

採用する AI 関連ツール

2026-04-22

採用する AI 関連ツール

このコンサルティング結果では、Qt 開発における AI 活用を、Qt に寄った支援、汎用的な実装支援、GUI テスト保守支援、IDE 連携補助に分けて整理する。そのうえで、主要 AI ツールとして `Qt AI Assistant`、`GitHub Copilot plugin for Qt Creator`、`Squish AI Assistant` を採用し、補足的な連携機能として `Qt Creator MCP server` を位置づける。

主要 AI ツール

Qt AI Assistant

採用理由:

- Qt / QML に寄った公式の開発支援であり、Qt 開発文脈に合った提案を期待できるため
- コード提案、レビュー、テスト補助、説明生成を 1 つの流れで扱いやすいため
- Qt 固有の API、設計、実装上の論点を補助する役割として整理しやすいため
- Qt 開発向けの AI 活用を顧客へ説明する際に、公式ツールとして提案しやすいため

期待する役割:

- 実装工程でのコード提案
- Qt / QML に寄ったレビュー支援
- 必要に応じた単体テストや補助コードの生成支援
- コードや設計内容の説明支援

GitHub Copilot plugin for Qt Creator

採用理由:

- Qt Creator 上で日常的なコード補完や定型コード入力を補助できるため
- Qt 専用ではないが、実装速度向上に直接効きやすいため
- Qt AI Assistant を補完する汎用的な入力支援として有効なため
- 既に利用経験のある開発者が多く、導入イメージを共有しやすいため

期待する役割:

- 定型コードや補助コードの補完
- 実装中の反復入力の削減
- テストコードや小さな補助処理の入力支援

Squish AI Assistant

採用理由:

- GUI テスト保守、失敗解析、ログ読解の負荷を下げられるため
- `Squish for Qt` を使う前提で、テストコードの改善や保守効率向上に寄与するため
- GUI テスト運用で起こりやすい属人化を緩和する補助として有効なため
- テスト自動化そのものではなく、テスト運用の生産性向上策として位置づけやすいため

期待する役割:

- Squish スクリプトの改善支援
- テスト失敗原因の説明補助
- runner / server ログの要約
- GUI テスト保守時の補助

補足的な連携機能

Qt Creator MCP server

採用理由:

- AI そのものではないが、Qt Creator の `build`、`run`、`debug`、`Issues` 確認を AI と連携できるため
- 実装工程と検証工程のローカル反復を短くできるため
- Qt Creator 上の文脈を利用した補助として価値があるため
- 今後の拡張が進めば、Qt 開発支援の接続点として重要度が高まる可能性があるため

期待する役割:

- 実装工程での `build` / `debug` 補助
- 検証工程での再現と切り分け補助
- `Issues` 情報を用いた修正ループ支援

留意点:

- Qt Creator MCP server は AI そのものではなく、補足的な連携機能として扱う
- 現時点では新しく、基本機能中心のため、限定的な補助用途に留める
- CI やリモート実行の中心には置かず、ローカルの Qt Creator 文脈利用に限定する

位置づけの整理

- Qt AI Assistant
 - Qt 開発に寄った公式 AI 支援
- GitHub Copilot plugin for Qt Creator
 - Qt Creator 上での汎用的な実装補完支援
- Squish AI Assistant
 - GUI テスト保守と失敗解析の AI 支援
- Qt Creator MCP server
 - AI と Qt Creator をつなぐ補足的な連携機能

採用方針

この構成では、Qt に寄った支援は Qt AI Assistant、日常的な実装補完は GitHub Copilot plugin for Qt Creator、GUI テスト運用支援は Squish AI Assistant に担わせる。一方で Qt Creator MCP server は、AI と IDE をつなぐための補助的な連携機能として扱い、主役ではなく実装・検証ループを支える補足要素として位置づける。